

CyberSOC Platform

Specification Expansion Document

Kubernetes Deployment | Known Limitations | Technical Debt Register

Document Version 1.0 Expansion

Classification Internal Technical

Date 2026-08-24

Status Production Ready

Table of Contents

1. Executive Summary

Overview of expansion scope and objectives

2. Kubernetes Deployment

Complete K8s architecture, Helm charts, operators

2.1 Architecture Overview

Microservices deployment topology

2.2 Helm Chart Structure

Chart organization and values

2.3 Custom Operators

CyberSOC-specific K8s operators

2.4 Deployment Workflows

CI/CD integration and GitOps

2.5 Security Hardening

Pod security, network policies, secrets

3. Known Limitations Register

Consolidated limitations with severity

3.1 Limitation Categories

Classification framework

3.2 Complete Register

All documented limitations

3.3 Mitigation Strategies

Risk treatment approaches

4. Technical Debt Register

Tracking template and format

4.1 Register Structure

Fields and metadata

4.2 Debt Categories

Classification taxonomy

4.3 Tracking Workflow

Review and remediation process

5. Implementation Roadmap

Phased improvement plan

1. Executive Summary

This document provides comprehensive expansions for three critical areas identified in the CyberSOC Platform specification audit. The original specification document encompasses 97 sections covering 41 distinct modules, representing one of the most complete Security Operations Platform specifications in the industry. However, the audit identified three specific areas requiring enhanced detail and formalized documentation structures to achieve production readiness.

Expansion Area	Original Coverage	Target State	Priority
Kubernetes Deployment	Brief mentions, Docker references	Complete Helm charts, operators, security hardening	CRITICAL
Known Limitations	Scattered throughout spec	Centralized register with severity classification	HIGH
Technical Debt Register	Listed as deliverable only	Complete tracking template with workflow	HIGH

Strategic Objectives: These expansions serve multiple strategic purposes within the CyberSOC development lifecycle. First, they provide implementation teams with actionable technical specifications rather than conceptual guidance alone. Second, they establish governance frameworks for tracking platform maturity and debt remediation over time. Third, they ensure consistency across distributed development teams by providing standardized templates and formats. Fourth, they support compliance requirements by documenting limitations transparently and establishing auditable tracking mechanisms for technical debt.

Intended Audience: This document targets platform architects responsible for deployment topology decisions, DevOps engineers implementing CI/CD pipelines, security teams validating production hardening, project managers tracking technical debt, and compliance officers requiring formal limitation documentation. Each section provides sufficient depth for its primary audience while maintaining cross-references for holistic understanding.

2. Kubernetes Deployment

The CyberSOC Platform requires a robust Kubernetes-native deployment strategy that addresses the unique challenges of security operations workloads. Unlike traditional applications, SOC platforms must handle high-volume event ingestion, real-time correlation processing, low-latency alert delivery, and stringent data isolation requirements. This section establishes the complete Kubernetes deployment architecture including Helm chart structure, custom operators, security hardening specifications, and operational workflows.

2.1 Architecture Overview

The CyberSOC Kubernetes deployment follows a microservices architecture pattern optimized for security operations workloads. The architecture separates concerns across multiple layers: data ingestion, stream processing, analytics, storage, presentation, and platform services. Each layer scales independently based on workload characteristics, enabling cost-effective operation while maintaining performance SLAs.

Layer	Components	Replica Strategy	Resource Class
Ingestion	Collector Gateway, Parser Pool, Normalizer	HPA (2-50 pods)	Compute-optimized
Streaming	Kafka Cluster, Stream Processor	StatefulSet (3+ nodes)	Memory-optimized
Analytics	Correlation Engine, ML Pipeline, UEBA	HPA (3-100 pods)	GPU-capable
Storage	Elasticsearch, PostgreSQL, Redis, Graph DB	StatefulSet (varies)	Storage-optimized
API Layer	REST Gateway, WebSocket, GraphQL	HPA (3-30 pods)	Balanced
Presentation	Frontend SPA, Report Generator	Deployment (2-10 pods)	Standard
Platform	Auth Service, Config Manager, Scheduler	Deployment (fixed)	Standard

Namespace Strategy: The deployment utilizes a multi-namespace approach to enforce isolation boundaries and simplify access control. The primary namespaces include: cybersoc-system (platform services), cybersoc-ingest (data collection), cybersoc-analytics (processing engines), cybersoc-storage (databases), cybersoc-api (external interfaces), and cybersoc-ai (ML/AI components). For MSSP deployments, each tenant receives an isolated namespace following the pattern cybersoc-tenant-{id}. Network policies restrict inter-namespace communication to explicitly allowed paths, preventing lateral movement even if a single component is compromised.

2.2 Helm Chart Structure

The CyberSOC platform uses a parent-child Helm chart structure that balances configurability with operational simplicity. The root chart orchestrates all subcharts while allowing independent versioning and deployment of individual components. This structure supports multiple deployment scenarios from single-cluster development environments to multi-region

production deployments with complex affinity rules.

Chart Directory Structure:

```
cybersoc/
├── Chart.yaml          # Parent chart metadata
├── values.yaml         # Default configuration
├── values-prod.yaml    # Production overrides
├── charts/
│   ├── ingress/       # NGINX/Envoy ingress controller
│   ├── collector/     # Log/event collectors
│   ├── kafka/         # Apache Kafka cluster
│   ├── elasticsearch/ # Search engine cluster
│   ├── postgresql/    # Primary database
│   ├── redis/         # Cache layer
│   ├── api-gateway/   # REST/WebSocket gateway
│   ├── correlation/   # SIEM correlation engine
│   ├── soar/          # SOAR automation
│   ├── ui/            # Frontend application
│   ├── ai-copilot/    # AI/LLM services
│   └── monitoring/    # Prometheus/Grafana stack
├── templates/
│   ├── _helpers.tpl
│   ├── namespace.yaml
│   ├── network-policies.yaml
│   └── rbac.yaml
├── docs/
└── values-schema.yaml
```

Critical Configuration Parameters: The parent values.yaml defines global configuration that propagates to all subcharts. Key parameter groups include: image registry credentials (pull secrets, private repo URLs), resource quotas (default limits, max limits, resource classes), feature flags (module enable/disable toggles), external service connections (database URLs, API keys via Kubernetes secrets), scaling thresholds (HPA min/max, target CPU/memory utilization), and tenant configuration (MSSP mode, isolation level, per-tenant limits). Environment-specific overlays modify these defaults without altering the base charts.

Parameter	Type	Default	Description
global.imageRegistry	string	ghcr.io/cybersoc	Container image registry URL
global.storageClass	string	premium-ssd	Kubernetes StorageClass for PVCs
ingress.enabled	bool	true	Enable ingress controller deployment
kafka.replicaCount	int	3	Kafka broker replica count
elasticsearch.clusterSize	int	3	Elasticsearch data nodes
mssp.enabled	bool	false	Enable MSSP multi-tenant mode
aiCopilot.enabled	bool	true	Enable AI Copilot services
monitoring.prometheus.enabled	bool	true	Deploy Prometheus stack

2.3 Custom Kubernetes Operators

CyberSOC deploys custom Kubernetes operators to manage platform-specific resources that exceed the capabilities of standard Kubernetes primitives. These operators implement the control pattern for security operations, handling complex lifecycle management, automated healing, and domain-specific orchestration that generic controllers cannot provide. Each operator follows the Operator SDK framework and implements reconciliation logic appropriate to its managed resource type.

Operator Name	CRD	Responsibility	Reconciliation Cycle
DetectionRule	DetectionRule	Manages Sigma/YARA rule lifecycle: validation, deployment, versioning, metrics collection	30s (on change)
Playbook	SOARPlaybook	Orchestrates playbook execution: trigger handling, step management, state persistence	10s (active)
Tenant	MSSTenant	Provisions tenant namespaces, RBAC, resource quotas, network isolation	60s (on change)
CorrelationPipeline	CorrelationPipeline	Manages streaming correlation topologies, rule distribution, scaling events	15s (on change)
ThreatIntelFeed	TIPLFeed	Synchronizes threat intelligence feeds: TAXII polling, IOC enrichment, expiration	300s (scheduled)
BackupJob	BackupSchedule	Manages backup workflows: snapshot creation, retention enforcement, restore verification	Per schedule
ComplianceScan	ComplianceAssessment	Executes compliance checks: control mapping, evidence collection, gap reporting	Per schedule

Implementation Standards: All operators must implement standardized status conditions reflecting the reconciliation state: Available (resource ready for use), Degraded (partial functionality), Error (reconciliation failed), and Progressing (undergoing changes). Each operator emits Kubernetes events for significant state transitions and exposes Prometheus metrics for observability. Operators must handle graceful shutdown, support leader election for high-availability deployments, and implement proper finalization logic to prevent resource leakage on deletion.

2.4 Deployment Workflows and CI/CD Integration

The Kubernetes deployment integrates with GitOps methodologies using ArgoCD or Flux CD as the continuous delivery agent. All declarative configurations reside in version-controlled repositories with approval gates for production changes. The CI pipeline builds container images, runs security scanning, executes integration tests, and pushes validated artifacts to the registry. The CD pipeline synchronizes desired state from Git to clusters, handling progressive rollouts and automatic rollback on health check failures.

Deployment Pipeline Stages:

- 1. Source Stage:** Pull Helm charts and configuration from Git repository. Validate chart structure, lint templates, check values schema conformance.

- 2. **Build Stage:** Compile containers using multi-stage Dockerfiles. Embed build metadata (commit SHA, timestamp, builder identity). Sign images with cosign or equivalent.
- 3. **Security Scan Stage:** Execute Trivy for vulnerability detection. Run SAST with Semgrep or similar. Check SBOM against known vulnerability databases. Fail on CRITICAL/HIGH CVEs.
- 4. **Test Stage:** Deploy to ephemeral kind cluster. Run integration tests against real dependencies. Execute load tests for ingestion components. Validate detection rule parsing.
- 5. **Promote Stage:** Push validated images to release registry. Update Helm chart version in manifest repo. Trigger ArgoCD sync for target environment.
- 6. **Monitor Stage:** Observe rollout progress. Verify health endpoints. Check error rates against baseline. Auto-rollback if degradation detected.

2.5 Kubernetes Security Hardening

Security hardening for the CyberSOC Kubernetes deployment follows defense-in-depth principles, addressing host-level, container-level, network-level, and application-level attack surfaces. Given that the platform processes sensitive security telemetry and may contain access credentials for integrated systems, the hardening posture exceeds typical enterprise standards. All configurations align with CIS Kubernetes Benchmark and NSA Kubernetes Hardening Guidance where applicable.

Pod Security Standards: All pods run with restricted Pod Security Standards enforcing: read-only root filesystem (except where write cache required), non-root user execution (UID > 10000), drop ALL capabilities adding only explicit needs, seccomp profiles set to RuntimeDefault or strict, and no new privilege escalation. Resource limits are mandatory for both requests and limits to prevent Noisy Neighbor attacks. Containers processing untrusted input (parsers, normalizers) run with additional sandboxing using gVisor or Kata Containers.

Network Policy Framework: Default-deny network policies apply to all namespaces. Explicit allow rules permit required communication paths: ingress controllers to API pods, collector pods to Kafka brokers, correlation engines to Elasticsearch, frontend to API gateway. East-west traffic between analytics components requires mTLS encryption via service mesh (Istio or Linkerd). Egress filtering restricts outbound connections to approved destinations (package repositories, threat intel feeds, notification services). Network policy auditing runs continuously to detect policy drift.

Secrets Management: Kubernetes secrets store only encrypted payloads. External Secrets Operator synchronizes from HashiCorp Vault or AWS Secrets Manager, enabling rotation without pod restarts. Sealed Secrets or Mozilla Sops handle Git-committed secrets for bootstrap scenarios. Audit logging captures all secret access with 90-day retention. Automatic secret rotation configured for: database credentials (30 days), API keys (90 days), TLS certificates (365 days), and encryption keys (per policy).

Category	Control	Implementation	Validation
----------	---------	----------------	------------

Host	Kernel hardening	CIS-compliant AMI/node config	kube-bench scans
RBAC	Least privilege	Role-based access, no cluster-admin	rbac-review audits
Network	Segmentation	Default-deny + explicit allows	NetworkPolicy coverage
Secrets	Encryption at rest	External Secrets + envelope encryption	Secret rotation tests
Supply Chain	Image integrity	Signed images, admission controller	Cosign verification
Runtime	Threat detection	Falco/Falcosidekick alerts	Alert response testing

3. Known Limitations Register

This section consolidates all known limitations identified during the CyberSOC Platform specification audit into a formal register. Each limitation is classified by category, assessed for severity impact, assigned ownership for resolution tracking, and linked to mitigation strategies. The register serves as the authoritative source of truth for platform capabilities boundaries, supporting accurate customer communication, sales engineering qualification, and development prioritization decisions.

3.1 Limitation Classification Framework

Limitations are organized into six categories reflecting different aspects of platform capability constraints. This classification enables targeted mitigation planning and helps stakeholders quickly identify limitations relevant to their concerns. Each category has a defined severity scale and standard remediation approach.

Category	Description	Examples	Typical Mitigation
Functional	Features not yet implemented or explicitly out of scope	Native EDR, email security actioning	Integration with third-party, roadmap commitment
Performance	Known scalability boundaries or latency constraints	EPS limits, query timeouts at scale	Architecture optimization, hardware sizing guide
Integration	Unsupported connectors or protocols	Legacy SIEM APIs, proprietary formats	Custom connector development, middleware
Environmental	Deployment constraints or prerequisites	Air-gapped requirements, OS support	Alternative deployment patterns, documentation
Compliance	Certification gaps or regulatory limitations	FedRAMP, specific regional requirements	Certification roadmap, compensating controls
Operational	Runbook gaps or procedure limitations	Disaster recovery RTO for petabyte data	Enhanced procedures, tooling investment

3.2 Complete Limitations Register

The following table presents all known limitations extracted from the specification document, supplemented with structured metadata for tracking and resolution management. Severity levels follow the standard classification: Critical (blocks production deployment), Major (significant functional gap), Moderate (workaround available), Minor (cosmetic or edge-case impact). Each entry includes the original specification reference for traceability.

ID	Limitation	Category	Severity	Status
LIM-001	No native EDR capabilities; relies on third-party integrations	Functional	Major	Accepted
LIM-002	Cannot assume Internet connectivity for updates/intelligence	Environmental	Major	Mitigated
LIM-003	OT/ICS module is optional; passive monitoring only	Functional	Moderate	Accepted

LIM-004	No automatic disruptive actions against industrial systems	Functional	By Design	Accepted
LIM-005	Active scanning requires explicit authorization	Operational	Moderate	Documented
LIM-006	Integration claims require actual API testing	Integration	Critical	Process Defined
LIM-007	AI-generated detections need validation before production	Functional	Major	Mitigated
LIM-008	Rare behavior does not equal malicious behavior (UEBA)	Functional	By Design	Accepted
LIM-009	Compliance mappings do not auto-grant certification	Compliance	Critical	Documented
LIM-010	Dashboard is not a complete feature indicator	Operational	Minor	Documented
LIM-011	Mobile high-risk actions require strong auth	Functional	Moderate	Accepted
LIM-012	Algerian localization requires ongoing regulatory updates	Compliance	Moderate	Process Defined
LIM-013	Plugin sandbox requires controlled environment	Environmental	Major	In Progress
LIM-014	Chaos engineering tests require careful scheduling	Operational	Moderate	Documented
LIM-015	Purple team simulations must be authorized	Operational	By Design	Accepted

3.3 Mitigation Strategies by Severity Level

Each severity level triggers a standardized response protocol ensuring consistent handling across the organization. Critical limitations block production deployments until mitigated or formally accepted by executive stakeholders with documented risk acknowledgment. Major limitations require active remediation tracks with defined milestones and regular status reviews. Moderate limitations should have documented workarounds and scheduled improvement items. Minor limitations enter the backlog for future consideration.

Severity	Response Required	Timeline	Escalation Path
Critical	Immediate mitigation or deployment hold; executive risk acceptance if accepted	0-30 days	CISO → CEO
Major	Active sprint allocation; workaround documentation; customer communication	30-90 days	Engineering Lead → VP Engineering
Moderate	Backlog prioritization; knowledge base article; training material update	1-2 quarters	Product Manager → Product Director
Minor	Product backlog entry; address in natural roadmap cadence	Next 2 releases	Team Lead → Product Manager
By Design	Document rationale; communicate proactively; no remediation planned	N/A	Architecture Review Board

4. Technical Debt Register

Technical debt represents the implied cost of additional rework caused by choosing an easy or fast solution now instead of using a better approach that would take longer. In the context of the CyberSOC Platform, technical debt accumulates through architectural shortcuts, deferred refactoring, incomplete test coverage, documentation gaps, and dependency management issues. This section establishes the formal register structure, categorization taxonomy, and operational workflow for managing technical debt throughout the platform lifecycle.

4.1 Register Structure and Fields

The technical debt register follows a standardized schema ensuring consistent capture, assessment, and tracking of all debt items. Each entry contains mandatory fields for identification and assessment, plus optional fields for contextual information. The register exists as both a machine-readable format (YAML/JSON) for automation integration and human-readable format (this document) for review sessions.

Field	Type	Required	Description
debt_id	string	Yes	Unique identifier (format: DEBT-NNN)
title	string	Yes	Concise description of the debt item
category	enum	Yes	Classification from taxonomy (see 4.2)
component	string	Yes	Affected module/subsystem
description	text	Yes	Detailed explanation of debt origin and impact
severity	enum	Yes	Critical/Major/Minor/Trivial
interest_rate	enum	Yes	High/Medium/Low (velocity of debt growth)
principal_effort	string	Yes	Estimated effort to fix (story points/days)
accrued_date	date	Yes	When debt was introduced
owner	string	Yes	Team/person responsible for remediation
status	enum	Yes	Identified/Planned/In Progress/Resolved/Waived
target_date	date	No	Planned remediation completion date
blocking	string[]	No	List of debt_ids this item blocks
blocked_by	string[]	No	List of debt_ids blocking this item
risk_if_unresolved	text	No	Consequences of not addressing this debt

4.2 Technical Debt Taxonomy

Technical debt in the CyberSOC Platform is categorized into eight distinct types, each with characteristic causes, symptoms, and remediation approaches. This taxonomy enables consistent classification, supports trend analysis over time, and guides appropriate remediation strategies. Items may span multiple categories when applicable, with the primary category determining default handling procedures.

Category	Definition	Common Causes	Remediation Approach
----------	------------	---------------	----------------------

Code Debt	Suboptimal code quality, missing abstractions, duplicated logic	Time pressure, skill gaps, evolving requirements	Refactoring sprints, code review enforcement
Design Debt	Architectural shortcuts, anti-patterns, coupling issues	Premature optimization, unclear requirements	Architecture review, incremental redesign
Test Debt	Insufficient test coverage, flaky tests, missing assertions	Testing deprioritized, complex setup requirements	Test coverage targets, QA investment
Documentation Debt	Outdated or missing docs, API reference gaps	Documentation not valued, rapid iteration	Docs-as-code, automated generation
Infrastructure Debt	Configuration drift, outdated IaC, manual processes	Quick fixes, lack of automation investment	GitOps adoption, infrastructure refactoring
Dependency Debt	Outdated libraries, vulnerable packages, license issues	Update fear, compatibility concerns	Automated updates, dependency dashboard
Security Debt	Known vulnerabilities, weak crypto, permission issues	Security shortcuts, legacy compatibility	Security sprints, penetration testing
Performance Debt	Scalability bottlenecks, inefficient algorithms, N+1 queries	Prototype code in production, missing optimization	Profiling, load testing, optimization sprints

4.3 Tracking and Remediation Workflow

The technical debt management process operates on a bi-weekly cadence aligned with sprint boundaries, ensuring continuous visibility and steady remediation progress. The workflow defines clear states, transitions, and responsibilities for each debt item from identification through resolution or formal waiver. Automation supports the workflow through integration with issue trackers, CI/CD pipelines, and code review tools.

Workflow States and Transitions:

IDENTIFIED: Debt discovered through code review, architecture analysis, or operational incident. Requires initial assessment within one sprint. Transition: → PLANNED (accepted) or → WAIVED (rejected).

PLANNED: Debt accepted into backlog with owner assignment and effort estimation. Target date established. Transition: → IN PROGRESS (sprint allocation) or → WAIVED (priority change).

IN PROGRESS: Active remediation underway. Code changes in progress, tests being written, or infrastructure being updated. Transition: → RESOLVED (completion) or → PLANNED (rollback/pause).

RESOLVED: Remediation complete and verified. Tests passing, documentation updated, deployed to production. Item retained in register for historical analysis.

WAIVED: Formal decision to not remediate. Requires business justification and risk acceptance. Annual review mandatory to confirm waiver remains valid.

Sample Technical Debt Entries

ID	Title	Category	Interest	Effort	Status
DEBT-001	Monolithic parser component needs decomposition	Design	High	21 days	Planned
DEBT-002	SIEM correlation engine lacks unit tests (< 30% coverage)	Test	Medium	14 days	In Progress
DEBT-003	Legacy Elasticsearch client library (v6) incompatible with v8 features	Dependency	High	10 days	Planned
DEBT-004	API documentation auto-generation not implemented	Documentation	Low	7 days	Identified
DEBT-005	Helm values.yaml lacks schema validation	Infrastructure	Medium	3 days	In Progress
DEBT-006	Hardcoded encryption keys in configuration examples	Security	Critical	2 days	In Progress

5. Implementation Roadmap

This section outlines the recommended phased approach for addressing the expanded content areas. The roadmap balances quick wins against foundational improvements, ensuring that immediate operational needs are met while building toward long-term architectural goals. Timeline estimates assume a dedicated team of 3-5 engineers with appropriate domain expertise.

Phase	Duration	Deliverables	Success Criteria
Phase 1: Foundation	Weeks 1-4	· Helm chart scaffolding · Base operator framework · Limitation register initialization · Debt register template creation	Charts deploy successfully; registers populated from spec audit
Phase 2: Core Operators	Weeks 5-10	· DetectionRule operator · Playbook operator · Tenant operator (MSSP) · CorrelationPipeline operator	Operators pass integration tests; CRDs stable
Phase 3: Security Hardening	Weeks 11-14	· PodSecurityProfiles · NetworkPolicy matrix · Secrets management integration · CIS benchmark compliance	kube-bench score > 80%; zero privileged containers
Phase 4: CI/CD Maturity	Weeks 15-18	· ArgoCD/Flux integration · Image signing pipeline · Automated security scanning · Multi-environment promotion	Full GitOps workflow; < 30 min PR-to-production
Phase 5: Operational Excellence	Weeks 19-24	· Remaining operators (TI, Backup, Compliance) · Runbook library · Debt remediation sprint · Documentation completion	All registers current; debt trending downward

Next Steps: Upon approval of this expansion document, the recommended immediate actions include: (1) allocate Phase 1 resources and establish the working repository structure, (2) conduct a workshop with platform engineers to validate the Helm chart architecture against existing deployment patterns, (3) populate the initial limitation and debt registers through a collaborative specification review session, and (4) define success metrics and reporting cadence for ongoing tracking. Regular review of these registers should occur during monthly architecture forums, with executive summaries provided to leadership quarterly.